🧪 ASP.NET Core — Pratik Task Listesi

> **Seviye:** Orta–İleri | **Konu:** Getting Started · HTTP · Middleware · Routing
> Tüm görevler **Minimal API** yapısıyla yapılacak. Her görevi bitirince kodunu gözden geçirebilirsin.

✅ TASK 1 — Projeyi Sıfırdan Kur ve HTTP Temellerini Dene

Amaç: Boş bir ASP.NET Core Minimal API projesi oluştur, temel HTTP mekaniklerini elle yönet.

Alt Görevler:

- ☐ `dotnet new web` ile `StajyerAPI` adında yeni bir proje oluştur
- ☐ `launchSettings.json` içinde `http` profilini düzenle → port `5050` olarak ayarla
- ☐ Aşağıdaki 3 endpoint'i sadece `Program.cs` içinde tanımla (controller yok):
  - `GET /saglik` → `200 OK`, body: `"Sistem ayakta"`, Content-Type: `text/plain`
  - `GET /gizli` → **301 Redirect** → `/saglik`'e yönlendirsin (manuel `Response.Redirect` kullan)
  - `POST /yankilama` → isteğin `Content-Type` header'ını ve body uzunluğunu response'a yaz
- ☐ Postman veya `curl` ile her 3 endpoint'i test et, status code'ların doğru geldiğini kontrol et

> **İpucu:** `context.Response.StatusCode`, `context.Response.Headers`, `context.Request.ContentLength` — bunların hepsini `HttpContext` üzerinden erişebilirsin.

✅ TASK 2 — Middleware Pipeline'ını Tasarla

Amaç: Birden fazla katmanlı, sıralı middleware pipeline'ı kurarak `app.Use`, `app.Run`, `app.UseWhen` farkını pratikte göster.

Alt Görevler:

- ☐ **Geleneksel (Conventional) middleware** olarak `IstekGunlukcusu` sınıfını yaz:
  - Constructor'da `RequestDelegate _next` al
  - `Invoke(HttpContext)` metodunda; isteğin HTTP metodunu, path'ini ve timestamp'i konsola yaz
  - `InvokeAsync(HttpContext)` çağrıldıktan sonra response status code'unu da konsola yaz

- `await _next(httpContext)` çağırdıktan **sonra** response status code'unu da konsola yaz
- Extension metot ekle: `UseIstekGunlukcusu()`

- [ ] **IMiddleware tabanlı** `YetkiKontrolcusu` middleware'ini yaz:
  - `InvokeAsync(HttpContext context, RequestDelegate next)` uygula
  - Request header'ında `X-API-KEY` yoksa `401 Unauthorized` döndür ve pipeline'ı kes
  - Varsa `next(context)` ile devam et
  - `AddTransient<YetkiKontrolcusu>()` ile kaydet
- [ ] `Program.cs` pipeline sırası **kesinlikle** şu şekilde olsun:

```
IstekGunlukcusu → UseWhen → YetkiKontrolcusu → Endpoint
```

- [ ] `app.UseWhen` ile şu koşulu branch'e al: Query string'de `?debug=true` varsa ek bir `Use` bloğuyla `"[DEBUG MODU]"` prefix'i response'a ekle
- [ ] Pipeline kırılırsa (401) `IstekGunlukcusu`'nun response status kodunu hâlâ logladığını doğrula

> **Dikkat:** `IMiddleware` kullanıyorsan DI kaydı zorunlu. Conventional middleware'de DI kaydı gerekmez — farkı gör!

✅ TASK 3 — Gelişmiş Routing ve Kısıtlamalar

Amaç: `context.Request.RouteValues` **kullanmadan**, saf parametre binding ile güçlü rotalar yaz.

Alt Görevler:

- [ ] Aşağıdaki endpoint'leri `Program.cs`'e ekle. **RouteValues erişimi yasak**, parametreler doğrudan fonksiyon imzasına gelecek:

Rota	Kısıt	Parametre Tipi
`GET /urunler/{id:int:min(1)}`	id ≥ 1	`int id`
`GET /calisanlar/{ad:alpha:length(3,10)}`	sadece harf, 3–10 karakter	`string ad`
`GE Tra/porlar/{yil:int:min(2000)}/{donem:regex(^(Q1`	Q2	Q3
`GET /dosyalar/{*yol}`	catch-all	`string yol`

- [ ] `GET /urunler/{id}` endpoint'ine Query String desteği ekle:
  - `[FromQuery] string? kategori` parametresi ile `/urunler/5?kategori=elektronik`

`[FromQuery] string? kategori` parametresi ile `/urunler/5?kategori=elektronik` çalışsın

- ☐ Eğitim reposundaki `MonthsCustomConstraint` sınıfını **geliştir**:
  - Mevcut sınıfı kopyala ve `KartalConstraint` adıyla yeni bir sınıf oluştur
  - `"kartal"`, `"siha"`, `"akinci"` değerlerini kabul etsin (mock araç ismi doğrulaması)
  - `builder.Services.AddRouting(options => options.ConstraintMap.Add("kinsystem", typeof(KartalConstraint)))` ile kaydet
  - `GET /envanter/{arac:kinsystem}` endpoint'ini ekle

> **Hatırlatma:** Custom constraint'i `ConstraintMap`'e eklemezsen rota **404** döner, runtime hata vermez — bu tuzağa düşme!

✅ TASK 4 — Global Hata Yönetimi ve Statik Dosyalar

Amaç: Pipeline'ın en tepesine gerçek bir `GlobalErrorHandler` otur ve statik dosya servisini konfigüre et.

Alt Görevler:

- ☐ `GlobalErrorHandler` adında **conventional middleware** yaz:
  - try-catch bloğu ile tüm pipeline'ı sarmala
  - `context.Response.HasStarted` kontrolü yap — response başlamışsa loglayıp çık
  - Hata durumunda: `500`, `Content-Type: application/json`, body: `{"hata": "Sunucu patladı", "zaman": "..."}`
  - Extension metot: `UseGlobalErrorHandler()`
- ☐ `GET /patlat` endpoint'i ekle → içinde `throw new Exception("Test patlaması!")` yapsın
- ☐ Pipeline sırası:

```
UseGlobalErrorHandler → UseStaticFiles → (diğer middleware) → Endpoint'ler
```

- ☐ Projeye `wwwroot/` klasörü ekle, içine basit bir `index.html` koy
- ☐ `app.UseStaticFiles()` ekle, `/index.html` tarayıcıda açılsın
- ☐ **Bonus:** `wwwroot` yerine `PublicAssets` adında özel bir klasör oluştur, `StaticFileOptions` + `PhysicalFileProvider` ile onu da serve et; `/PublicAssets/test.txt` üzerinden erişilebilir olsun

---

✅ TASK 5 — Tüm Parçaları Birleştir: Gerçekçi Senaryo

Amaç: Önceki 4 task'taki her şeyi tek bir `Program.cs` altında birleştiren, gerçek hayata yakın

bir API iskelet yaz.

Senaryo:

Bir **İnsansız Hava Aracı (İHA) Envanter API'si** yazıyorsun. Yetkisiz erişim engellenecek, loglanacak, hatalar yakalanacak.

Alt Görevler:

- ☐ Pipeline (kesin sıra):

```
GlobalErrorHandler
  → UseStaticFiles
  → IstekGunlukcusu (her isteği logla)
  → UseWhen(?debug=true → debug header bas)
  → YetkiKontrolcusu (X-API-KEY kontrolü)
  → Endpoint'ler
```

- ☐ Endpoint'ler:
  - `GET /` → `wwwroot/index.html` veya `"IHA Envanter API v1"` dönsün
  - `GET /envanter` → sabit bir liste dönsün: `["TB2", "Akıncı", "SIHA"]` (JSON olarak)
  - `GET /envanter/{arac:kinsystem}` → ilgili aracın detayını dönsün (mock data yeterli)
  - `GET /envanter/{id:int:min(1)}` → id ile arama, `[FromQuery] bool? detayli` parametresi alsın
  - `GET /patlat` → exception fırlatsın, GlobalErrorHandler yakalamalı
- ☐ Tüm sayfanın pipeline'ı çalışır durumda olmalı:
  - `X-API-KEY` olmadan istek → `401`
  - Geçersiz rota → `404` (MapFallback ile özelleştir)
  - Exception → `500 JSON`
  - Static file → doğrudan servis

> **Son Test:** Postman collection oluştur, her senaryoyu çalıştır. Hangi middleware'in ne zaman devreye girdiğini console loglarından izle.

📊 Puan Tablosu (Kendini Değerlendirmek İçin)

Task	Konu	Zorluk
Task 1	Getting Started + HTTP	⭐⭐
Task 2	Middleware Pipeline	⭐⭐⭐
Task 3	Routing + Constraints	⭐⭐⭐⭐
Task 4	Error Handling + Static Files	⭐⭐⭐

Task 4	Error Handling + Static Files	⭐⭐⭐
Task 5	Tüm Konular Entegrasyonu	⭐⭐⭐⭐⭐